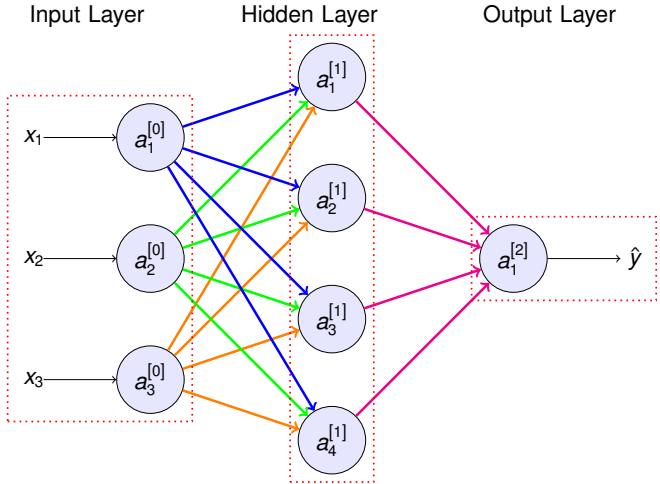
Two Layer Neural Network Architecture



Computing the Activations

For Hidden layer

$$
\begin{aligned}
z_1^{[1]} &= w_1^{[1]} a^{[0]} + b^{[1]} \\
a_1^{[1]} &= \sigma\left(z_1^{[1]}\right) \\
z_2^{[1]} &= w_2^{[1]} a^{[0]} + b^{[1]} \\
a_2^{[1]} &= \sigma\left(z_2^{[1]}\right) \\
z_3^{[1]} &= w_3^{[1]} a^{[0]} + b^{[1]} \\
a_3^{[1]} &= \sigma\left(z_3^{[1]}\right) \\
z_4^{[1]} &= w_4^{[1]} a^{[0]} + b^{[1]} \\
a_4^{[1]} &= \sigma\left(z_4^{[1]}\right)
\end{aligned}
$$

In the matrix form

$$
\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} = \underbrace{\begin{bmatrix} \dots & w_1^{[1]T} & \dots \\ \dots & w_2^{[1]T} & \dots \\ \dots & w_3^{[1]T} & \dots \\ \dots & w_4^{[1]T} & \dots \end{bmatrix}}_{4 \times 3} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{3 \times 1} + \underbrace{\begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}}_{4 \times 1}
$$

$$
z^{[1]} = \begin{bmatrix} w_1^{[1]\top} a^{[0]} + b_1^{[1]} \\ w_2^{[1]\top} a^{[0]} + b_2^{[1]} \\ w_3^{[1]\top} a^{[0]} + b_3^{[1]} \\ w_4^{[1]\top} a^{[0]} + b_4^{[1]} \end{bmatrix} \leftarrow 4 \times 1 \text{ matrix}
$$

$$
a^{[1]} = \sigma\left(z^{[1]}\right) \leftarrow 4 \times 1 \text{ matrix}
$$

Activations for Training Examples

Vectorizing for one training example

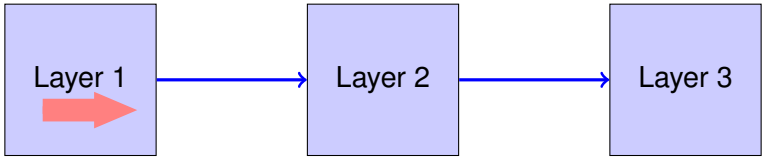
$$
\begin{aligned}
a^{[0]} &= X \\
Z^{[1]} &= W^{[1]} a^{[0]} + b^{[1]} \\
a^{[1]} &= \sigma\left(Z^{[1]}\right) \\
Z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\
a^{[2]} &= \sigma\left(Z^{[2]}\right) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$

Vectorizing for all training examples

$$
\begin{aligned}
\text{Example 1} &: X^{(1)} \longrightarrow a^{[2](1)} = \hat{y}^{(1)} \\
\text{Example 2} &: X^{(2)} \longrightarrow a^{[2](2)} = \hat{y}^{(2)} \\
\text{Example 3} &: X^{(2)} \longrightarrow a^{[2](3)} = \hat{y}^{(3)} \\
&\vdots \qquad\qquad\qquad \vdots \\
\text{Example m} &: X^{(m)} \longrightarrow a^{[2](m)} = \hat{y}^{(m)}
\end{aligned}
$$

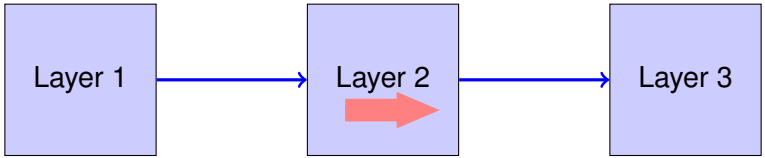
Neural Network Training - Forward Pass

Step 1: Compute Loss on mini-batch [Forward pass]



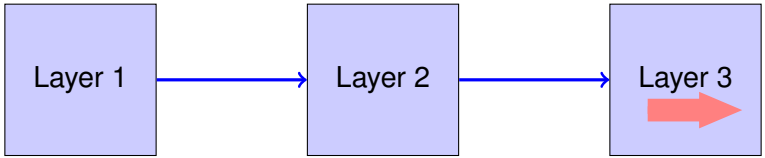
Neural Network Training - Forward Pass

Step 1: Compute Loss on mini-batch [Forward pass]



Neural Network Training - Forward Pass

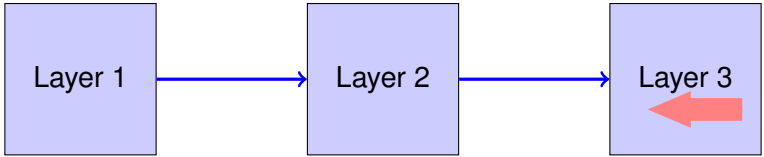
Step 1: Compute Loss on mini-batch [Forward pass]



Neural Network Training - Backward Pass

Step 1: Compute Loss on mini-batch [Forward pass]

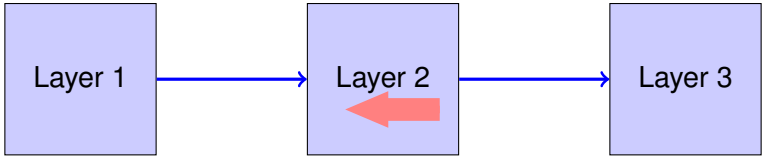
Step 2: Compute gradients wrt parameters [Backward pass]



Neural Network Training - Backward Pass

Step 1: Compute Loss on mini-batch [Forward pass]

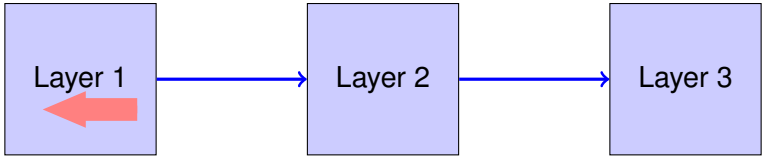
Step 2: Compute gradients wrt parameters [Backward pass]



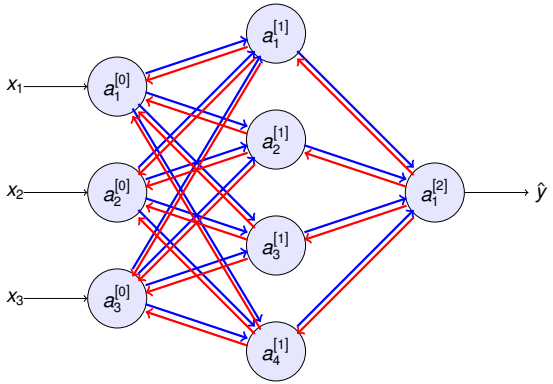
Neural Network Training - Backward Pass

Step 1: Compute Loss on mini-batch [Forward pass]

Step 2: Compute gradients wrt parameters [Backward pass]



Back Propagation of Cost Function



Weights	$W^{[1]} + b^{[1]}$	$W^{[2]} + b^{[2]}$
Gradients	$da^{[1]} + dz^{[1]}$	$da^{[2]} + dz^{[2]}$

Derivation of Gradients in Last Layer

$$z = w^\top x + b$$

$$a = \sigma(z)$$

$$L = (-y \log a) - (1 - y) \log(1 - a)$$

$$\frac{\partial L}{\partial a} = \frac{-y}{a} + \frac{1 - y}{1 - a}$$

$$dz = \frac{\partial L}{\partial z} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z} = \left(\frac{-y}{a} + \frac{1 - y}{1 - a}\right) a(1 - a) = a - y$$

$$dw = \frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w} = (a - y)x$$

Computing the Gradients

For one training example

$$dz^{[2]} = a^{[2]} - y$$
$$dw^{[2]} = dz^{[2]} \cdot a^{[1]T}$$
$$db^{[2]} = dz^{[2]}$$
$$dz^{[1]} = \left(w^{[2]T} dz^{[2]}\right) * \sigma'(z^{[1]})$$
$$dw^{[1]} = dz^{[1]} \cdot x^{\top}$$
$$db^{[1]} = dz1$$

For all training examples

$$dZ^{[2]} = A^{[2]} - Y$$
$$dW^{[2]} = \frac{1}{m}\left(dZ^{[2]} \cdot A^{[1]T}\right)$$
$$db^{[2]} = \frac{1}{m}\sum dZ^{[2]}$$
$$dZ^{[1]} = \left(W^{[2]T} dZ^{[2]}\right) * \sigma'(Z^{[1]})$$
$$dW^{[1]} = \frac{1}{m}\left(dZ^{[1]} \cdot X^{\top}\right)$$
$$db^{[1]} = \frac{1}{m}\sum dZ1$$